

System Design - Complete Course Notes

Based on Telusko's *System Design One Shot* (5 hr 5 min)

Compiled Study Notes

[How These Notes Are Organized](#)

[Part 1 — Foundations](#)

[1.0 Why Learn System Design at All?](#)

[1.1 What Is System Design?](#)

[1.2 Types of System Design](#)

[1.3 Web App vs Website vs Architecture](#)

[1.4 System Types: Monoliths vs Distributed Systems](#)

[1.5 Latency vs Throughput](#)

[1.6 Availability, Time & Consistency](#)

[1.7 Storing “Hot Data”](#)

[1.8 WebSockets & SSE \(Server-Sent Events\)](#)

[1.9 Scaling Techniques](#)

[1.10 Redundancy & Replication \(Preview\)](#)

[1.11 CAP Theorem \(Preview\)](#)

[1.12 Functional vs Non-Functional Requirements](#)

[1.13 Putting Part 1 Together: A Mental Checklist](#)

[Part 2 — Communication](#)

[2.1 What Happens When You Hit a URL? \(DNS Deep Dive\)](#)

[2.2 High-Level Client-Server Architecture](#)

[2.3 What Is an API?](#)

[2.4 REST APIs in Detail](#)

[2.5 Web Servers & Protocols \(Supporting Concepts\)](#)

[2.6 App Security Basics](#)

[Part 3 — Data Layer](#)

[3.1 SQL Databases](#)

[3.2 NoSQL Databases](#)

[3.3 Graph Databases](#)

[3.4 Caching](#)

[Part 4 — Scaling & Distribution](#)

[4.1 Load Balancers](#)

[4.2 Replication](#)

[4.3 Partitioning \(Sharding\)](#)

[4.4 CAP Theorem](#)

[Part 5 — Reliability & Operations](#)

[5.1 Synchronous vs Asynchronous Communication](#)

[5.2 Message Queues \(MQ\)](#)

[5.3 Fault Tolerance](#)

[5.4 Monitoring: Latency Percentiles](#)

[Part 6 — Case Study: Designing a Video Streaming App](#)

[6.1 Estimating Requirements](#)

[6.2 End-to-End Architecture](#)

[6.3 Why Each Piece Is There \(Concept Recap Mapped to This System\)](#)

[Appendix A — Worked Capacity-Estimation Practice](#)

[Appendix B — Interview Prep: Questions & Model Answers](#)

[Part 1 — Foundations](#)

[Part 2 — Communication](#)

[Part 3 — Data Layer](#)

[Part 4 — Scaling & Distribution](#)

[Part 5 — Reliability & Operations](#)

[Part 6 — Case Study Thinking](#)

[Appendix C — Bonus Practice: Applying the Toolkit to a New Problem](#)

[Design Problem: A URL Shortener \(e.g., bit.ly\)](#)

[Appendix D — Common Pitfalls & Anti-Patterns](#)

[Quick-Reference Glossary](#)

[How to Use These Notes for Interview Prep](#)

How These Notes Are Organized

This document follows the exact roadmap used in the course:

1. **Foundations** — What is System Design, Components, Data vs Compute Intensive, Functional & Non-Functional Requirements
2. **Communication** — DNS, APIs, REST in Detail
3. **Data Layer** — SQL, NoSQL, Graph DB, Caching
4. **Scaling & Distribution** — Load Balancing, Replication, Partitioning, CAP Theorem
5. **Reliability & Operations** — Sync/Async Communication, Message Queues, Fault Tolerance, Monitoring
6. **Case Study** — Designing a Video Streaming Application (YouTube-style)

Each section reconstructs the diagrams, examples and terms used on screen, expanded with enough explanation to be usable as standalone interview-prep notes.

Part 1 — Foundations

1.0 Why Learn System Design at All?

Before diving into the roadmap, it's worth being explicit about *why* this subject exists and why it dominates senior engineering interviews.

Writing code that works for **one user on your laptop** is a fundamentally different problem from writing a system that works for **ten million users spread across the world**, all hitting it at once, on unreliable networks, with hardware that fails randomly. System design is the discipline of bridging that gap. It sits one level above “can I write correct code” and asks “will this *survive contact* with real-world scale, failure, and growth?”

Three forces make this hard, and the entire course is essentially a tour of the tools we've invented to fight each of them:

- **Scale** — more users, more data, more requests than any single machine can serve.
- **Failure** — hardware dies, networks partition, humans make mistakes; nothing is 100% reliable.
- **Change** — requirements evolve, traffic patterns shift, and a design that worked at 1,000 users may collapse at 1,000,000 users if it doesn't have room to grow.

Every concept from here on — caching, replication, partitioning, load balancing, queues, CAP theorem — is a direct, practical answer to one or more of these three forces. Keep this frame in mind as you read; it turns a long list of buzzwords into a coherent story.

1.1 What Is System Design?

System Design is the process of defining the **architecture, components, modules, interfaces, and data** of a system so that it satisfies a given set of requirements — functionally and non-functionally (speed, scale, reliability).

The Motivating Story: A Small Shop

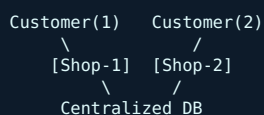
The course opens with a simple analogy instead of jargon: imagine a small shop billing customers manually.

- A customer arrives, the shopkeeper **understands the requirement, counts the cash, and prepares the receipt** — this whole process takes about **10 minutes per customer**.
- **Issue #1: The process is slow.** Two obvious fixes are proposed:
 - Train staff to count faster.
 - Train staff to type faster.

These are *local, short-term optimizations* — they only delay the real problem. As the shop grows and gets more customers (“traffic”), the same bottleneck reappears at a bigger scale. This is exactly the situation software systems face: a single process/server/database that worked for 10 users starts failing at 10,000 users.

From One Shop to a Centralized System

Picture two shop branches (S1, S2) both querying a **single centralized database**. Every request from every customer funnels into that one DB.



As **requests** and **traffic** increase, this central point becomes a bottleneck and a single point of failure — the seed of almost every system design problem discussed later (scaling, replication, partitioning, caching all exist to solve this).

Order → SMS Example (Async Thinking Introduced Early)

A second mini example: placing an **Order** triggers an **SMS** notification. Instead of the order flow waiting for the SMS to send (which is slow and unrelated to placing the order), the system pushes the “send SMS” task onto a **Message Queue (MQ)**, and a separate worker/service consumes it and sends the SMS. This is the first appearance of **asynchronous, decoupled** communication — a pattern revisited in depth in Part 5.

The Four Recurring “Worries” of Any System

Whenever we design a system, four questions keep coming back:

1. **How fast can we read the data?** → performance, caching, indexing
2. **How safely are we going to store the data?** → durability, replication, backups
3. **How many users can access it simultaneously?** → concurrency, scaling, load balancing
4. **What happens if a machine dies?** → fault tolerance, redundancy, failover

Almost the entire rest of the course is organized as answers to these four questions, expressed through a recurring set of **components/terms**:

- Databases
- Caching
- Replication
- Sharding (Partitioning)
- Consistency

1.2 Types of System Design

- **HLD — High-Level Design:** The 30,000-ft view. Focuses on overall architecture — which services exist, how they talk to each other, what databases and caches are used, how traffic flows. Diagrams here are boxes-and-arrows: client, load balancer, servers, databases, caches, queues.
- **LLD — Low-Level Design:** The zoomed-in view. Focuses on class design, object relationships, method signatures, design patterns, and how a *single service* is implemented internally (this course focuses primarily on HLD).

Types of Software Architecture Patterns

Beyond the HLD/LLD zoom level, it helps to know the common *architectural styles* a system can be built with — these recur constantly in both this course and real design interviews:

- **Layered (N-Tier) Architecture** — the system is split into horizontal layers (presentation, business logic, data access), each only talking to the layer directly below it. Simple to reason about; common in traditional enterprise apps.
- **Monolithic Architecture** — the entire application (UI, business logic, data access) is one deployable codebase (detailed in §1.4).
- **Microservices Architecture** — the application is split into many small, independently deployable services, each owning its own data (detailed in §1.4).
- **Event-Driven Architecture** — services communicate primarily by producing and consuming events (often via a message queue/broker, §5.2) rather than direct request/response calls — enables loose coupling and high scalability at the cost of harder end-to-end debugging.

- **Serverless / Function-as-a-Service (FaaS)** — individual functions run on-demand, fully managed by a cloud provider (e.g., AWS Lambda), scaling automatically to zero and up without the team managing servers directly. Best for spiky, event-triggered workloads.
- **Client-Server Architecture** — the foundational pattern underlying almost everything else in this course (§2.2): clients request, servers respond.

1.3 Web App vs Website vs Architecture

- A **website** is largely static/informational content.
- A **web app** is interactive, has business logic, state, user accounts, and typically a backend + database behind it.
- **Architecture** is the way all the moving pieces (frontend, backend, database, cache, queue, third-party services) are structured and connected to deliver the application's behavior.

1.4 System Types: Monoliths vs Distributed Systems

Aspect	Monolith	Distributed / Microservices
Codebase	Single deployable unit	Multiple independent services
Scaling	Scale the whole app together	Scale individual services independently
Failure impact	One bug can bring the whole app down	Failure can be isolated to one service
Complexity	Simple to build & deploy initially	Complex: networking, service discovery, distributed data
Best for	Small teams, early-stage products	Large teams, systems needing independent scaling

This tradeoff resurfaces later (Part 4) when the course discusses the disadvantage of **sticky sessions** forcing a monolithic-style dependency on one server, versus a microservices-friendly stateless design.

1.5 Latency vs Throughput

- **Latency** — the time taken for a single request to get a response (e.g., “this API responds in 200ms”).
- **Throughput** — the number of requests a system can process in a given time window (e.g., “this system handles 10,000 requests/second”).

A system can have low latency but low throughput (fast for one user, chokes under load), or high throughput with higher latency (handles huge volume, but each request is a bit slower). Good system design balances both according to the product's needs.

1.6 Availability, Time & Consistency

- **Availability** — the system is up and responding, even if some data returned is slightly stale.
- **Consistency** — every read returns the most recent write, no matter which node answers.
- **Time (Latency of consistency propagation)** — how quickly updates on one node become visible on others.

These three ideas are the seeds of the **CAP theorem**, formally covered in Part 4.

1.7 Storing “Hot Data”

Not all data is equal. **Hot data** is data that is accessed very frequently (a trending product, a popular video’s metadata, a celebrity’s profile). Because hot data is requested repeatedly, it is an ideal candidate for **caching** (Part 3) — keeping it in fast, in-memory storage close to the application instead of hitting the database every time.

1.8 WebSockets & SSE (Server-Sent Events)

For scenarios where the server needs to push updates to the client continuously (chat apps, live scores, notifications):

- **WebSockets** — a full-duplex, persistent connection: both client and server can send messages to each other at any time. Ideal for chat apps, multiplayer games, collaborative editing.
- **Server-Sent Events (SSE)** — a one-way channel where the server continuously pushes updates to the client over a single long-lived HTTP connection (client cannot send messages back over the same channel). Ideal for live feeds, notifications, stock tickers.

1.9 Scaling Techniques

- **Vertical Scaling (Scale Up)** — add more CPU/RAM/disk to the *same* machine. Simple, but has a hard ceiling and a single point of failure.
- **Horizontal Scaling (Scale Out)** — add *more machines* and distribute load across them (this is where Load Balancers, discussed in Part 4, become essential).

Horizontal scaling is generally preferred for internet-scale systems because it avoids the hardware ceiling of vertical scaling and improves fault tolerance (losing one of many machines is survivable; losing your one giant machine is not).

Types of Auto-Scaling

Once you’ve decided to scale horizontally, *when* do new machines get added or removed? Two common strategies:

- **Reactive (Threshold-Based) Auto-Scaling** — new instances are added automatically when a metric (CPU usage, request queue length, memory) crosses a defined threshold, and removed when load drops back down. Simple and widely used (e.g., AWS Auto Scaling Groups).
- **Predictive / Scheduled Scaling** — capacity is increased *ahead of time* based on known traffic patterns (e.g., scaling up an e-commerce system an hour before a flash sale starts) rather than reacting after load has already increased.

Scaling the Database Layer Specifically

Scaling isn't only about application servers — the database layer scales differently:

- **Read Scaling** — add read replicas (§4.2) to spread read traffic across multiple nodes while writes still go to one primary.
- **Write Scaling** — requires partitioning/sharding (§4.3) or a multi-leader/leaderless replication model, since a single-writer database eventually becomes the write bottleneck no matter how many read replicas you add.

1.10 Redundancy & Replication (Preview)

Redundancy means keeping extra copies of critical components (servers, data) so that if one fails, another can take over. **Replication** is the mechanism of keeping those copies of data in sync. Covered in full depth in Part 4.

1.11 CAP Theorem (Preview)

A distributed system can only fully guarantee **two out of three** of: **C**onsistency, **A**vailability, **P**artition tolerance. Covered in full depth with the CA-system example in Part 4.

1.12 Functional vs Non-Functional Requirements

This is one of the very first steps of any real system design exercise — before drawing any boxes, clarify:

Functional Requirements

What the system must do — the actual features. Example used in the course: designing an **E-commerce app like Amazon**:

- Users can search and browse products
- Users can add items to cart and check out
- Users can track orders
- Sellers can list products

Non-Functional Requirements

How well the system must do it — the quality attributes:

- **Scalability** — can it handle growth in users/data?
- **Availability** — is it accessible with minimal downtime?
- **Latency** — how fast are responses?
- **Consistency** — how up-to-date must the data be across replicas?
- **Durability** — is data safe from loss?
- **Security** — is data/access protected?

Data-Intensive vs Compute-Intensive Systems

- **Data-Intensive** systems are bottlenecked by the *volume, complexity, or speed* of data movement/storage — e.g., a social media feed, an e-commerce catalog. The hard problems are storage, indexing, caching, replication.
- **Compute-Intensive** systems are bottlenecked by *processing power* — e.g., video transcoding, ML model inference, scientific simulations. The hard problems are parallelization, CPU/GPU allocation, job scheduling.

Recognizing which category a system falls into (or a mix of both, as in the streaming-app case study in Part 6) determines where to invest design effort. A data-intensive system pushes you to think hardest about Part 3 and Part 4 (storage, caching, replication, partitioning); a compute-intensive system pushes you to think hardest about horizontal scaling of *workers* and job scheduling, exactly as seen in the transcoding stage of the Part 6 case study.

1.13 Putting Part 1 Together: A Mental Checklist

Before any diagram gets drawn in a real design session, a strong candidate silently runs through:

1. What is this system *for*, in one sentence? (functional requirements)
2. What are the 2–3 non-functional requirements that matter most *for this specific system*? (Not every system needs five-nines availability; not every system needs sub-100ms latency. Prioritizing is itself a skill.)
3. Is this primarily data-intensive, compute-intensive, or both?
4. Roughly how many users / requests / how much data are we talking about? (even rough numbers change the design — see the estimation approach in Part 6.1)
5. Which of the four recurring “worries” (§1.1) is the biggest risk for *this* system?

This checklist is the thread that connects Part 1’s abstractions to Parts 2–6’s concrete tools.

Key Takeaways — Part 1

- System design is about satisfying functional *and* non-functional requirements, not just “making the feature work.”
- Every scaling problem eventually traces back to one root cause: a **single point** (server, database, process) that can’t keep up with growing demand.
- Horizontal scaling is generally favored over vertical scaling for internet-scale systems because it avoids a hardware ceiling and improves fault tolerance.
- Classify a system as data-intensive vs compute-intensive early — it changes where your design effort should go.

Part 2 — Communication

2.1 What Happens When You Hit a URL? (DNS Deep Dive)

When a user types `telusko.com` into a browser, a multi-step resolution process converts that human-readable domain into a machine-usable **IP address**. DNS (Domain Name System) is often called “the phonebook of the internet” — and like a real phonebook, it’s organized as a hierarchy so that no single server has to know every domain in the world.

The Full Request/Response Sequence for Getting an IP Address

Every one of these numbered arrows is a real network round trip. Nothing is skipped — this is the complete path a query takes before your browser has an IP address to connect to:

```
(1) Browser → DNS Resolver
    "What is the IP
    for telusko.com?"

(2) DNS Resolver → Root Server
    "Who handles .com?"

(3) Root Server → DNS Resolver
    "Ask the .com TLD Server
    at 40.20.30.10"

(4) DNS Resolver → TLD Server (.com)
    "Who is authoritative
    for telusko.com?"

(5) TLD Server → DNS Resolver
    "Ask the Authoritative
    Name Server for telusko.com"

(6) DNS Resolver → Authoritative Name Server
    "What is the A record
    for telusko.com?"

(7) Authoritative Name Server → DNS Resolver
    "19.23.38.41"

(8) DNS Resolver → Browser
    "19.23.38.41"
    (cached locally using the record's TTL)

(9) Browser → 19.23.38.41
    Opens a TCP/TLS connection directly
    to the server and sends the HTTP request
```

Step-by-Step Resolution (as drawn in the course, mapped onto the sequence above)

1. **(Steps 1)** The browser asks the **ISP**, which forwards the query to a **DNS Resolver** (also called a Recursive Resolver) — this is a single **recursive** query from the browser’s

point of view (§ below): the browser just wants the final answer and doesn't want to talk to Root/TLD/Authoritative servers itself.

2. **(Steps 2-3)** The DNS Resolver asks a **Root Server** ("Where do I find `.com` domains?"). The root server doesn't know the final IP — it only replies with the address of the **TLD (Top-Level Domain) server** responsible for `.com` (example IP shown in the course: `40.20.30.10`).
3. **(Steps 4-5)** The resolver then asks that **TLD server** ("Who manages `telusko.com`?"). The TLD server also doesn't know the final IP — it replies with the address of the **Authoritative Name Server** for that specific domain.
4. **(Steps 6-7)** The resolver finally asks the **Authoritative Name Server**, which is the *only* server in this whole chain that actually holds `telusko.com`'s real DNS records. It returns the final IP address (example shown in the course: `19.23.38.41`).
5. **(Step 8)** The Resolver hands that IP back to the browser, and caches it locally (based on the record's TTL) so the *entire* Root → TLD → Authoritative walk doesn't need to repeat for the next request to the same domain.
6. **(Step 9)** Only now does the browser actually open a connection — a TCP handshake (and TLS handshake, for HTTPS) directly to `19.23.38.41` — and send its first real HTTP request.

Each of steps 2 through 7 is an **iterative** exchange from the resolver's perspective: the resolver keeps getting told "I don't know the final answer, but here's who to ask next," and it walks down the hierarchy one referral at a time, until step 6-7 finally produces the real answer.

Types of DNS Servers (Roles in the Hierarchy)

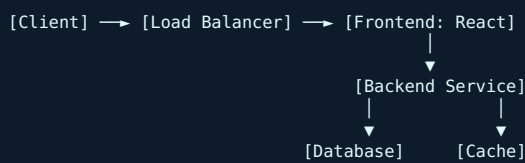
- **DNS Resolver (Recursive Resolver)** — the entry point for a query; usually run by your ISP or a public provider (e.g., Google's `8.8.8.8`, Cloudflare's `1.1.1.1`). It does the work of walking the hierarchy on the client's behalf (steps 2-7 above) and caches results for future queries.
- **Root Server** — the top of the DNS hierarchy (13 logical root server clusters worldwide, identified `a.` through `m.root-servers.net`). It doesn't know the final IP; it only knows which TLD server to ask next.
- **TLD (Top-Level Domain) Server** — manages a specific domain suffix (`.com`, `.org`, `.in`, `.io`, etc.) and knows which Authoritative Name Server is responsible for any given domain under that suffix.
- **Authoritative Name Server** — the final source of truth for a specific domain; it actually stores that domain's DNS records (A, CNAME, MX, etc., below) and returns the real answer.

Recursive vs. Iterative Queries

- **Recursive Query** — the client asks the resolver *once* and expects a final, complete answer; the resolver itself does all the work of walking Root → TLD → Authoritative on the client's behalf (this is what a browser/OS does — it asks its configured DNS resolver recursively).
- **Iterative Query** — the resolver asks each server in the chain (Root, then TLD, then Authoritative) and each one responds with “I don't know the final answer, but ask this other server next” — this is how the *resolver itself* talks to Root/TLD/Authoritative servers, referral by referral, until it reaches the answer.

2.2 High-Level Client-Server Architecture

Once the browser has the server's IP, a typical modern web architecture looks like:



- **Frontend (FE)** — e.g., a React app — renders UI and talks to the backend via APIs.
- **Backend (BE)** — business logic, talks to the database and cache.
- Mobile clients (e.g., **Swift** for iOS) hit the *same* backend APIs as the web frontend — a key reason to design APIs as a clean, client-agnostic contract.

2.3 What Is an API?

An **API (Application Programming Interface)** is a contract that lets two software components talk to each other without knowing each other's internal implementation. The client sends a **request**; the server returns a **response**.

Types of APIs

Type	Full Form	Data Format	Notes
REST	Representational State Transfer	JSON (typically)	Lightweight, stateless, most common for web/mobile APIs
SOAP	Simple Object Access Protocol	XML	Strict, contract-based (WSDL), heavier, common in legacy enterprise systems
GraphQL	Graph Query Language	JSON	Client specifies <i>exactly</i> which fields it needs in a single query, avoiding the over-fetching/under-fetching common with fixed REST endpoints; one endpoint instead of many
gRPC	Google Remote Procedure Call	Protocol Buffers (binary)	High-performance, strongly-typed, built on HTTP/2; common for fast internal service-to-service

			communication in microservices
WebSocket API	—	Any (often JSON)	Full-duplex, persistent connection for real-time, bidirectional communication (§1.8) — chat, live collaboration, multiplayer

The course visually contrasts REST/JSON against SOAP/XML — showing an XML payload being converted to JSON and back to illustrate why REST + JSON became the default choice for modern APIs: JSON is lighter-weight and maps naturally to objects in most programming languages, while XML/SOAP carries more overhead and stricter schemas.

Choosing between them in practice: REST remains the default for public-facing web/mobile APIs because of its simplicity and cacheability. GraphQL is favored when clients have very different data needs (e.g., a mobile app needing a lean payload vs. a web dashboard needing a rich one) from the same backend. gRPC is favored for internal, latency-sensitive microservice-to-microservice calls where both ends are services you control. WebSocket APIs are reached for specifically when the server needs to push real-time updates rather than the client always initiating the request.

2.4 REST APIs in Detail

HTTP Methods (CRUD Mapping)

HTTP Method	Purpose	Example
GET	Read data	GET /users → list all users; GET /users/1 → get user with id 1
POST	Create data	POST /api/v3/users
PATCH	Partially update data	PATCH /users/{id} with body {"username": "ak"}
DELETE	Remove data	DELETE /users/{id}

Anatomy of a Full Request (as demonstrated on screen)

```
POST /api/v3/users
Content-Type: application/json

{
  "fname": "Akshay",
  "lname": "Agarwal",
  ...
}
```

- The **URL path** identifies the resource (/api/v3/users — note the API **version**, v3, baked into the path; a common REST convention for backward compatibility).
- The **Content-Type** header (application/json) tells the server how to parse the body.
- The **request body** carries the actual payload for POST / PATCH requests.

Requests & Responses Example

```
GET /users      → 200 OK  [ {...}, {...}, {...} ]
GET /users/1    → 200 OK  { "id": 1, "fname": "Akshay", ... }
```

- **Status codes** communicate the outcome: `200` = success, `4xx` = client error (e.g., `404` not found), `5xx` = server error.
- A `DELETE /users/{id}` call typically returns a confirmation (e.g., `"deleted"`) rather than the resource body.

REST API Design Principles Recap

- **Statelessness** — every request carries all the information the server needs; the server does not store client session state between requests (this becomes very important again in Part 4's discussion of *sticky sessions* and JWT).
- **Resource-oriented URLs** — nouns, not verbs (`/users/1`, not `/getUser?id=1`).
- **Use HTTP methods and status codes semantically**, instead of inventing your own protocol on top of HTTP.

2.5 Web Servers & Protocols (Supporting Concepts)

Rounding out the communication layer, a few protocol-level pieces sit underneath every API call shown above:

- **HTTP/HTTPS** — the application-level protocol carrying every REST request/response. HTTPS wraps HTTP in TLS encryption, which is why the course's API examples (and any real production API) must run over HTTPS: it protects credentials, tokens (like the JWT from Part 4), and payloads from being read or tampered with in transit.
- **TCP** — the underlying transport protocol HTTP is built on; it guarantees ordered, reliable delivery of bytes between client and server, at the cost of connection setup overhead (the TCP handshake) — part of why techniques like HTTP keep-alive and connection pooling matter for high-throughput APIs.
- **A Web Server's job** (e.g., Nginx, Apache, or an application server like Node/Express, Spring Boot) is to accept incoming HTTP connections, route them to the right application code, and return a response — often also handling concerns like TLS termination, static file serving, compression, and acting as a **reverse proxy** in front of application instances (frequently paired directly with the Load Balancer role from Part 4).

2.6 App Security Basics

Security is a non-functional requirement (§1.12) that touches almost every layer already discussed:

- **Authentication** — proving *who* you are (login, credentials).
- **Authorization** — proving *what* you're allowed to do once authenticated.
- **JWT** (previewed here, detailed in Part 4.1) is a common mechanism for carrying authentication/authorization claims statelessly across requests and services.

- **HTTPS everywhere** — encrypting data in transit (see §2.5).
- **Input validation** — never trust client-supplied data (query params, request bodies) without validating/sanitizing it server-side, to prevent injection attacks.
- **Rate limiting** — protecting APIs from abuse or accidental overload by capping how many requests a client can make in a given window (conceptually related to the Load Balancer’s job of protecting backend servers, Part 4.1).

Types of Authentication Mechanisms

- **Session-Based Authentication** — after login, the server creates a session record (often in memory or a shared store like Redis) and gives the client a session ID cookie; every subsequent request looks up that session server-side. Simple, but stateful — this is exactly what drives systems toward sticky sessions (§4.1) unless the session store itself is centralized/shared.
- **Token-Based Authentication (JWT)** — after login, the server issues a self-contained, signed token that encodes the user’s identity/claims; the client sends it on every request, and any server can verify it independently without a shared session store. Stateless, and the natural fit for horizontally scaled, load-balanced systems (§4.1).
- **OAuth 2.0 / OpenID Connect** — a standardized *delegation* protocol that lets a user grant a third-party app limited access to their data on another service (e.g., “Sign in with Google”) without sharing their password directly with the third party. Commonly layered on top of token-based auth.
- **API Keys** — a simpler mechanism (a static secret string sent with each request) typically used for service-to-service or third-party developer access rather than end-user login, often paired with rate limiting per key.

Types of Rate Limiting Algorithms

- **Fixed Window** — count requests in fixed time buckets (e.g., per minute); simple but allows bursts right at window boundaries.
- **Sliding Window** — smooths this out by considering a rolling time window instead of hard resets.
- **Token Bucket** — a bucket refills with tokens at a steady rate; each request consumes a token, allowing short bursts up to the bucket’s capacity while enforcing a steady average rate. Widely used in production API gateways.
- **Leaky Bucket** — requests are processed at a constant, steady output rate regardless of burstiness of arrival, “leaking” out at a fixed rate — prioritizes smoothing traffic over allowing bursts.

Key Takeaways — Part 2

- DNS resolution is a hierarchical lookup: Resolver → Root → TLD → Authoritative Name

Server, and it's heavily cached at every layer.

- A typical modern architecture separates Frontend, Backend, Database, and Cache, with a Load Balancer at the edge.
- REST + JSON is the default choice for most public/mobile APIs today because it's lightweight and stateless; SOAP + XML persists mainly in legacy enterprise integrations.
- HTTP methods should map semantically to CRUD operations, and status codes should communicate outcomes clearly.
- Security (HTTPS, auth, validation, rate limiting) is not an afterthought — it's a non-functional requirement baked into the communication layer from day one.

Part 3 — Data Layer

3.1 SQL Databases

Table Basics

A relational database organizes data into **tables**, where each **row** is a record and each **column** is an attribute.

Users

id	fname	lname	phone_no
1	Akshay	Agarwal	9876543210

Constraints

Constraints enforce data integrity at the database level:

- **UNIQUE** — no two rows can share the same value in that column (e.g., no two users with the same phone number).
- (Other standard constraints beyond what's shown: `PRIMARY KEY`, `NOT NULL`, `FOREIGN KEY`, `CHECK`.)

Relationships & JOINS

Relational databases model relationships between tables and use **JOIN** operations to combine rows from multiple tables based on a related column.

- **One-to-Many (1-N)** — e.g., one `Blog` has many `Posts`, and one `Post` has many `Comments`. Demonstrated with a `Users` table joined against related child records.

Users

id	fname
1	Akshay
2	Gaurav

1—N

Posts

id	user_id	title
1	1	"Blog A"
2	1	"Blog B"

- The **Blog → Post → Comments** hierarchy is used to illustrate nested one-to-many relationships, a very common real-world schema pattern (also applies to e.g. YouTube's Video → Comments in Part 6).

When to Reach for SQL

The course frames SQL's sweet spot clearly:

SQL → Payments, transactions, consistency.

SQL databases (with ACID transactions) are the right default whenever correctness and consistency matter more than raw write-scale — financial transactions, order processing, anything where you cannot afford a partial or inconsistent write.

3.2 NoSQL Databases

NoSQL → Scale, speed.

NoSQL databases relax strict relational structure and often relax strict consistency in exchange for **horizontal scalability** and **high write/read throughput**. They fit naturally with unstructured or semi-structured data, and with systems that need to scale out across many nodes (social feeds, catalogs, logs, session stores, IoT data).

Common NoSQL categories (implied by the SQL/NoSQL contrast and revisited via Graph DB below):

- **Document Stores** (e.g., MongoDB, Couchbase) — store data as JSON/BSON-like documents, where each document can have a different shape. Ideal for semi-structured data like user profiles, content catalogs, or CMS content, where a rigid schema would be limiting.
- **Key-Value Stores** (e.g., Redis, DynamoDB, Memcached) — the simplest NoSQL model: a unique key maps directly to a value (which can be a string, blob, or complex object). Extremely fast lookups, ideal for caching (§3.4), session storage, and configuration data.
- **Wide-Column Stores** (e.g., Cassandra, HBase) — data is organized into rows and dynamic columns grouped into “column families”; optimized for very high write throughput and queries over huge datasets spread across many nodes. Common in time-series logging, IoT telemetry, and messaging history at scale.
- **Graph Databases** (e.g., Neo4j) — relationship-centric data, covered in depth in §3.3.
- **Time-Series Databases** (e.g., InfluxDB, TimescaleDB) — purpose-built for data points indexed by time (server metrics, sensor readings, stock prices), with storage and query engines optimized for time-range queries and aggregations (like the P50/P90/P99

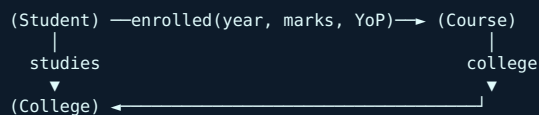
latency data discussed in §5.4).

Choosing Among NoSQL Types

Need	Best-Fit Type
Flexible, evolving schema for records (profiles, catalog items)	Document Store
Simplest possible fast lookups (cache, sessions)	Key-Value Store
Massive write throughput across many nodes	Wide-Column Store
Modeling and querying relationships/connections	Graph Database
Metrics/telemetry indexed by time	Time-Series Database

3.3 Graph Databases

Graph databases model data as **Nodes** (entities) and **Edges** (relationships), where both nodes and edges can carry **properties**.



- Example schema shown: **Student** — **studies** → **College**, **Student** — **enrolled** → **Course** (with edge properties like `year`, `marks`, `YoP`).
- **Primary use case**: finding patterns and relationships in highly connected data — commonly **used by data scientists** for recommendation engines, fraud detection, social network analysis.
- **Query languages** mentioned: **Gremlin**, **SparQL**, **Cypher** (Cypher is Neo4j’s query language; Gremlin is the Apache TinkerPop graph traversal language; SPARQL queries RDF-based graph/triple stores).

3.4 Caching

What Is a Cache?

A **cache** is fast, typically in-memory storage that helps in **loading resources faster**, sitting between the client/application and slower backing stores (database, disk, network calls). Poor cache design directly hurts business performance — the course explicitly frames this as a **business** concern: *poor performance* → *lost users/revenue*.

Where Caches Live



- **Client-side cache** — stored on the user’s device/browser (e.g., caching static assets, API responses locally) to avoid a network round trip entirely.
- **Server-side cache** — sits between the backend and the database.

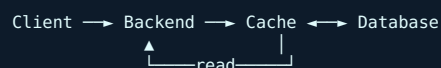
- **CDN (edge) cache** — geographically distributed caches close to end users, covered in depth in the Part 6 case study (§6.2); serves static/media content without a round trip to the origin server at all.
- **Database-level cache** — many databases (and database drivers) maintain their own internal query/buffer caches, an additional layer beneath the application-level caches described above.

Types of Cache-Population Strategies

Beyond *where* a cache lives, it also matters *how* it gets populated and kept in sync with the database — four standard strategies:

- **Cache-Aside (Lazy Loading)** — the application code is responsible for checking the cache, and on a miss, loading from the database and populating the cache itself. This is effectively what the Read-Through pattern below describes, made explicit in application code rather than handled transparently by the caching layer.
- **Read-Through Cache** — the *cache itself* (not the application) sits in front of the database and transparently loads from the DB on a miss, so the application only ever talks to the cache. Detailed next.
- **Write-Through Cache** — every write goes to the cache *and* the database together, keeping them always in sync at the cost of slightly slower writes.
- **Write-Behind (Write-Back) Cache** — writes go to the cache first and are asynchronously flushed to the database later, giving very fast writes at the risk of data loss if the cache fails before flushing.
- **Write-Around Cache** — writes go directly to the database, *bypassing* the cache; the cache is only populated on a subsequent read (read-through). Useful when written data is unlikely to be re-read soon, avoiding cluttering the cache with write-only data.

Read-Through Cache (RTC) Pattern



The **Read-Through Cache** pattern:

1. Backend receives a read request.
2. Backend checks the **cache** first.
3. **Cache Hit** — Cache → Backend directly. This is the **faster retrieval** path (data was already sitting in the cache).
4. **Cache Miss** — Backend → Cache → DB → Cache (populate) → Backend. This is the **slower** path since the database has to be queried, and the result is written back into the cache for next time.

Cache Invalidation

Whenever underlying data changes, the cached copy must eventually be updated or removed. Common strategies:

- A **TTL (time-to-live)** expiry on each cache entry, after which it's automatically dropped and re-fetched on the next request.
- **Explicit invalidation** at write time — as soon as the backend writes new data, it also deletes or updates the corresponding cache entry (used naturally by write-through).

Cache Eviction Policies

Since cache memory is limited, old/unneeded entries must be evicted to make room for new ones:

1. **LRU — Least Recently Used:** evict the entry that hasn't been *accessed* for the longest time. (Illustrated with an "Apple" product page — if it hasn't been viewed in a while, it's the first to be evicted when space is needed.)
2. **MRU — Most Recently Used:** evict the entry that was *most recently* accessed. Counter-intuitive, but useful in specific access patterns (e.g., when older items are statistically more likely to be re-accessed than the one you just used) — the course flags this with an **AP**-style tradeoff note, i.e., it's a niche policy chosen for specific access patterns rather than a general default.
3. **LFU — Least Frequently Used:** evict the entry with the lowest total access *count*, regardless of how recently it was touched. Better than LRU when some items are consistently popular over long stretches of time (recency doesn't predict future use as well as overall frequency does).
4. **FIFO — First In, First Out:** evict the oldest entry *by insertion time*, regardless of how often or recently it's been accessed. Simple to implement, but ignores actual usage patterns entirely.
5. **Random Replacement (RR):** evict a randomly chosen entry when space is needed. Cheapest to implement (no bookkeeping of access order/frequency needed) and surprisingly effective for some uniform-access workloads, though generally outperformed by LRU/LFU for real-world skewed access patterns.

Choosing an eviction policy in practice: LRU is the sensible default for most caches (web sessions, API response caching) because real-world access patterns usually have temporal locality (recently viewed items tend to be viewed again soon). LFU is preferred when popularity is stable over long periods (e.g., caching the metadata of consistently trending videos). FIFO and Random are chosen mainly for their simplicity/low overhead when the workload doesn't have a strong recency or frequency skew.

Key Takeaways — Part 3

- Choose SQL when transactional correctness matters (payments, orders); choose NoSQL when you need to scale horizontally with high throughput (feeds, catalogs, logs).
- Graph databases shine when the *relationships* between entities are the primary thing you're querying, not just the entities themselves.
- Caching is one of the highest-leverage performance tools available — it turns slow database round trips into fast in-memory lookups, but it introduces the new problem of keeping cache and source-of-truth data in sync (invalidation).
- Eviction policies (LRU, MRU, LFU, FIFO) decide *what to remove* when a fixed-size cache fills up; the right policy depends on your access pattern.

Part 4 — Scaling & Distribution

4.1 Load Balancers

A **Load Balancer (LB)** sits in front of a pool of servers and distributes incoming requests across them so no single server is overwhelmed.

Core Responsibilities

1. **Choosing the server** — deciding *which* backend instance should handle each incoming request.
2. **Health checks** — continuously pinging backend servers to make sure they're alive; unhealthy servers are automatically removed from the rotation.

Types of Load Balancers (by OSI Layer)

- **Layer 4 (L4) Load Balancer** — operates at the transport layer, routing based purely on IP address and port, without inspecting the actual content of the request. Very fast and protocol-agnostic, but can't make routing decisions based on, say, the URL path or HTTP headers.
- **Layer 7 (L7) Load Balancer** — operates at the application layer, meaning it can read the actual HTTP request (URL path, headers, cookies) and route accordingly — e.g., sending `/api/*` to one service and `/static/*` to another. More flexible, slightly higher overhead than L4.

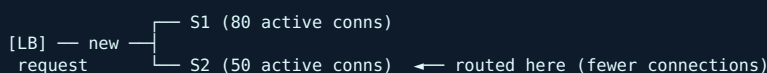
Types of Load Balancers (by Deployment)

- **Hardware Load Balancers** — dedicated physical appliances (e.g., F5), very high throughput, expensive, common in large on-prem data centers.
- **Software Load Balancers** — run as software on commodity servers or as a managed cloud service (e.g., Nginx, HAProxy, AWS ELB/ALB) — cheaper, more flexible, and the default choice for most modern cloud-native systems.

- **DNS Load Balancing** — the DNS server itself returns different IP addresses for the same domain to different clients (round-robin at the DNS level, §2.1), providing a coarse first layer of distribution before traffic even reaches a dedicated LB.

Load Balancing Algorithms

- **Round Robin** — requests are handed to servers in rotating order, one after another, looping back to the start; simplest baseline algorithm, works well when all servers have similar capacity and requests have similar cost.
- **Weighted Round Robin** — shown on screen as an LB splitting requests between two servers (S2, S3) in an **11-20** vs **9-12** style skewed ratio, illustrating that not all servers need to get an equal share (e.g., a more powerful server can be weighted to receive more traffic).
- **Least Connections Algorithm** — the LB checks how many active connections each server currently holds (e.g., Server-1 has 80 active connections, Server-2 has 50) and routes the *new* request to whichever server currently has **fewer active connections**. This adapts better than pure round-robin when requests have very different processing times.
- **IP Hash** — the client's IP address is hashed to consistently determine which server handles their requests, giving a simple form of session affinity (§ below) without needing a shared session store.
- **Least Response Time** — routes to whichever server currently has both a low active-connection count *and* the fastest recent response time, combining two signals instead of one.



Sticky Sessions (Session Affinity)

When a load balancer routes *all* requests from a given user to the *same* backend server for the duration of their session (so any server-side session state, e.g., login state stored in server memory, remains consistent):



Major disadvantage: it hurts scaling.

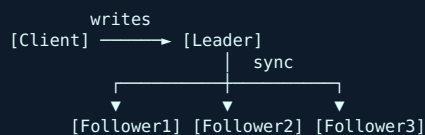
- If S1 goes down, that user's session data is lost, and re-balancing traffic evenly becomes harder because users are "pinned" to specific servers.
- It works against **microservices** philosophy and is more of a **monolithic/legacy-app** pattern, since modern distributed systems want any server to be able to handle any request statelessly.

The modern alternative: JWT (JSON Web Tokens). Instead of the server remembering session state, the server issues a signed token (JWT) to the client after login. The client sends this token on every request; *any* backend server can independently verify it (since it's self-contained and signed) — removing the need to pin a user to one specific server. This directly supports horizontal scaling and stateless microservices architecture.

4.2 Replication

Replication means maintaining multiple copies of the same data across different nodes/servers, primarily to improve **durability** (data safety) and **availability** (survive node failure). There are three broad models, differing in *how many* nodes are allowed to accept writes:

Leader-Follower (Master-Slave) Replication

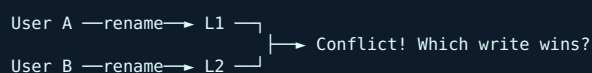


- All **writes** go to the **Leader**.
- The Leader **synchronizes** data to its **Followers**.
- **Followers are always updated** (kept in sync with the leader) and typically serve **read** traffic, spreading out read load.
- **If the Leader fails**, one of the Followers is promoted to become the new Leader, so the system keeps functioning.
- **Disadvantages:** replication lag (followers can briefly be behind the leader), and the failover process (detecting leader failure + promoting a follower) adds complexity and a brief window of reduced consistency/availability.

Multi-Leader Replication & Write Conflicts

When **more than one leader** can accept writes (useful for multi-region setups so users write to their nearest region), you introduce the possibility of **conflicts** — two leaders each writing to the *same piece of data* independently.

Example from the course: two users each try to **rename** the same record concurrently, connected to two different leaders (L1, L2):



Resolving such conflicts is a genuinely hard distributed-systems problem — common strategies (not verbatim from the video, but standard practice) include **last-write-wins** (by timestamp), **application-level merge logic**, or **CRDTs** (conflict-free replicated data types).

Leaderless Replication (Quorum-Based)

The third model removes the concept of a “leader” entirely: **any** node can accept a read or write, and consistency is instead enforced through **quorums** — requiring a minimum number of nodes to acknowledge a write (or agree on a read) before it’s considered successful.

- A common configuration is expressed as **N/W/R**: **N** = total replicas, **W** = replicas that must acknowledge a write, **R** = replicas that must respond to a read. As long as $W + R > N$, every read is guaranteed to overlap with the most recent write on at least one node, giving a tunable consistency/availability tradeoff.
- Used by systems like **Amazon DynamoDB** and **Cassandra** — a natural fit for **AP**-leaning systems (§4.4) that prioritize always being able to accept writes/reads even during partial node failures, accepting that conflicting writes may need reconciliation later (similar in spirit to the multi-leader conflict problem above).

Summary comparison:

Model	Writes Accepted By	Consistency	Complexity	Typical Use
Leader-Follower	One leader only	Strong (on leader)	Low-Medium	Most relational databases, read-heavy apps
Multi-Leader	Several designated leaders	Eventual, with conflicts	Medium-High	Multi-region apps needing local writes
Leaderless (Quorum)	Any node	Tunable via N/W/R	High	Massive-scale, high-write-availability systems

4.3 Partitioning (Sharding)

As data grows, a single database node cannot hold (or serve) everything efficiently.

Partitioning splits data across multiple nodes.

The Naive Approach and Its Problem

1. **Naive idea:** Copy the *entire* dataset to another node.
2. **Problem:** What if the data is huge? Copying everything to every node doesn’t solve the storage problem, and it makes searching / any other performance-sensitive operation slower as data keeps growing (e.g., illustrated by escalating sizes: 100GB → 200GB → 100TB — a single node clearly cannot keep scaling vertically forever).

The real solution is to **split (shard)** the data — each node holds only a *portion* of the total dataset, rather than a full copy.

Types of Partitioning (Sharding) Strategies

- **Range-Based Partitioning** — data is split by ranges of a key (e.g., user IDs 1–

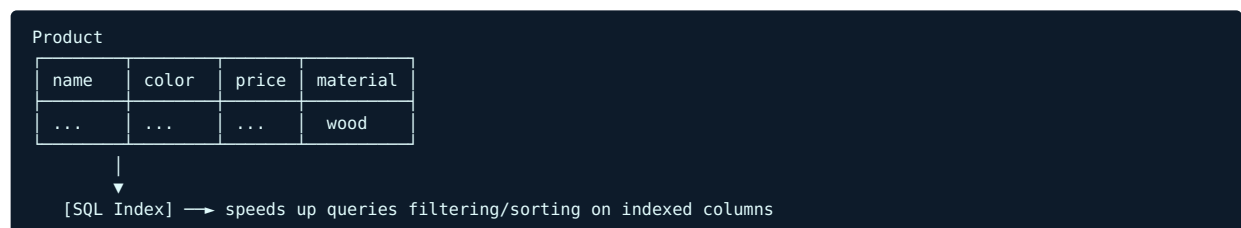
1,000,000 on Node A, 1,000,001–2,000,000 on Node B). Simple and makes range queries efficient, but can create **hot spots** if certain ranges (e.g., newest users) get disproportionately more traffic.

- **Hash-Based Partitioning** — a hash function is applied to the shard key, and the result determines which node stores the data. Distributes data much more evenly than range-based partitioning, but makes range queries (e.g., “all users created this week”) harder since related keys are scattered across nodes.
- **Geographic (Directory-Based) Partitioning** — data is split by region/location (e.g., EU users’ data stored in EU data centers) — useful for data residency/compliance requirements and for placing data close to the users who access it most.
- **Consistent Hashing** — an advanced hash-based scheme where both nodes and keys are placed on a conceptual “ring”; when a node is added or removed, only the keys immediately adjacent to it on the ring need to move, instead of re-shuffling the entire dataset (as plain hash-mod-N partitioning would require). Widely used in distributed caches and NoSQL stores (e.g., DynamoDB, Cassandra) specifically because it minimizes data movement during scaling events.

Indexes

Once data is large and spread out, **finding** the right rows/documents quickly becomes its own problem — solved with **indexes**.

Example schema shown:



An **index** is an auxiliary data structure (commonly a B-Tree) built on one or more columns that lets the database jump directly to matching rows instead of scanning the entire table — dramatically speeding up read/query performance at the cost of extra storage and slightly slower writes (since indexes must be updated on every insert/update).

Types of Indexes

- **Primary Index** — automatically built on a table’s primary key; guarantees uniqueness and is the fastest way to look up a single row.
- **Secondary Index** — built on any non-primary-key column (e.g., the `material` column in the Product example) to speed up queries that filter/sort on that column instead of the primary key.
- **Composite (Multi-Column) Index** — built across multiple columns together (e.g., `color + price`), speeding up queries that filter on that specific combination — but not

necessarily helpful for queries on just one of those columns in a different order.

- **B-Tree Index** — the default index structure in most relational databases; keeps data sorted and allows efficient range queries (`price > 100`) as well as exact-match lookups.
- **Hash Index** — optimized purely for exact-match lookups (`WHERE id = 5`), typically faster than a B-Tree for that specific case, but unable to support range queries at all.
- **Full-Text Index** — specialized for searching within large text fields (e.g., searching product descriptions for keywords), used by search-oriented systems (like Elasticsearch) as well as some relational databases' built-in full-text search.

4.4 CAP Theorem

Formally: in the presence of a **network partition (P)**, a distributed system must choose between **Consistency (C)** and **Availability (A)** — you cannot have all three simultaneously.

- **C — Consistency**: every node returns the same (latest) data.
- **A — Availability**: every request gets a (non-error) response, even if it isn't the absolute latest data.
- **P — Partition Tolerance**: the system keeps working even when network communication between nodes breaks down.

Since network partitions are a fact of life in any real distributed system, P is essentially non-negotiable — so in practice, the real-world choice is between **CP** and **AP** systems.

CA Example (as drawn in the course)

```
[Node A] —sync→ [Node B]
      stale data risk if sync is delayed
```

A system prioritizing **C + A** keeps data **Consistent** and **Available** but has to sacrifice full **Partition tolerance** — meaning during a network split it either has to stop serving some requests (breaking A) or serve older, **stale data** until nodes resynchronize. The course visualizes this stale-data risk with a simple two-node sync diagram: if node sync lags, a node might briefly answer with outdated information.

Practical takeaway used throughout the industry:

- **CP systems** (e.g., traditional relational databases, HBase, MongoDB in certain configurations) — refuse to answer rather than return stale/conflicting data. Good for banking, inventory counts.
- **AP systems** (e.g., Cassandra, DynamoDB) — always answer, possibly with slightly stale data, and reconcile later (“eventual consistency”). Good for social feeds, shopping carts, product catalogs, where a slightly-stale response is better than no response.

Consistency Models Beyond “Strong vs Eventual”

For deeper interview discussions, it helps to know CAP's C/A tradeoff isn't binary — there's a spectrum:

- **Strong consistency** — every read reflects the latest write, everywhere, immediately (hardest to scale).
- **Eventual consistency** — replicas converge to the same value *eventually*, if no new writes occur (easiest to scale, used heavily in AP systems).
- **Causal consistency** — operations that are causally related are seen in the same order by everyone, even if unrelated operations may be seen in different orders (a useful middle ground, e.g., for comment threads where a reply must never appear before the comment it replies to).

Key Takeaways — Part 4

- Load balancers solve the “many servers, one entry point” problem; the algorithm you choose (round robin, weighted, least-connections) should match your traffic's characteristics.
- Sticky sessions trade scalability for simplicity — prefer stateless authentication (JWT) so any server can handle any request.
- Replication protects against data loss and server failure; Leader-Follower is simple but has a single write bottleneck, Multi-Leader scales writes but introduces conflict resolution complexity.
- Partitioning (sharding) is what lets storage scale horizontally instead of hitting a hardware ceiling; indexes are what keep partitioned (and non-partitioned) data fast to query.
- CAP theorem is a lens for making an explicit, informed tradeoff under network partitions — not a checklist to “solve,” but a reminder that you cannot have perfect consistency and perfect availability simultaneously during a partition.

Part 5 — Reliability & Operations

5.1 Synchronous vs Asynchronous Communication

Synchronous (Sync)

The caller **waits** for the callee to finish and respond before proceeding.

- **When to use Sync:** When correctness and immediate confirmation matter, and later steps genuinely **depend** on the result.
- **Example from the course: a Finance App.** Money-movement operations need **security** and have a hard **dependency** on confirmation (you must know immediately whether a transaction succeeded) — so these are done synchronously.

Asynchronous (Async)

The caller **fires the request and moves on** without waiting (“**Fire & Forget**”); the work is completed independently, often notified back via a callback, event, or simply not needing a response at all.

- **Use cases shown: Broadcasting, Email, SMS/WhatsApp** notifications — none of these need to block the main flow of the application; the user doesn’t need to wait for an SMS to be *sent* before their app screen continues.

	Sync	Async
Caller blocks?	Yes	No
Best for	Payments, critical confirmations	Notifications, emails, logging, analytics
Coupling	Tighter	Looser (decoupled)

5.2 Message Queues (MQ)

A **Message Queue** decouples the producer of work from the consumer of work. The producer drops a message onto the queue and moves on; one or more consumers process messages independently, at their own pace.

Push-Based Requests

```
Producer →push→ [ 5 | 4 | 2 | 2 | 1 ] → Consumer(s)
                (Queue)
```

In a **push-based** model, the queue (or a broker) actively pushes new messages to available consumers as they arrive, rather than consumers having to repeatedly ask “is there anything new?” (polling).

Types of Messaging Patterns

- **Point-to-Point (Queue) Model** — each message is consumed by exactly **one** consumer, even if multiple consumers are listening (they compete for messages). Used when a task should be done exactly once (e.g., “process this one order,” “transcode this one video segment,” directly matching the Part 6 case study’s worker pool).
- **Publish-Subscribe (Pub/Sub) Model** — each message is broadcast to **every** subscriber interested in that topic/channel, not just one. Used for fan-out notifications (e.g., “a new video was uploaded” needs to notify the transcoding service, the notification service, *and* the analytics service simultaneously).

	Point-to-Point	Pub/Sub
Consumers per message	Exactly one	All subscribers
Typical tool	Task/job queues (e.g., SQS, RabbitMQ queues)	Event/topic brokers (e.g., Kafka topics, SNS)
Best for	Work distribution across a worker pool	Broadcasting an event to many independent services

Use Cases

- **Async workflows** — offloading slow or non-critical work (sending emails/SMS, as in Part 1's order example) from the main request path.
- **Decoupling** — producer and consumer don't need to know about each other or be online at the same time; if the consumer is temporarily down, messages simply wait in the queue.
- **Analytics** — event data (clicks, views, purchases) can be pushed onto a queue and processed by analytics pipelines asynchronously, without slowing down the user-facing request.

This is the same pattern used heavily in the Part 6 case study, where uploaded videos are pushed to a queue for transcoding workers to pick up (a point-to-point pattern — each video segment is transcoded by exactly one worker).

5.3 Fault Tolerance

Fault tolerance is a system's ability to keep operating correctly even when some component fails.

Types of Faults

- **Human Fault** — faults introduced by people (misconfiguration, bad deploys, accidental deletion). The course characterizes human faults as **unpredictable** and therefore **unreliable** to guard against with any single fixed rule — they require process-level safeguards (code review, staged rollouts, backups, permissions) rather than a purely technical fix.
- **Hardware Fault** — faults from physical infrastructure (disk failure, server crash, network hardware failure). Framed as **powerful** in the sense that a single hardware failure can take down a large blast radius if there's no redundancy — which is exactly why **replication** (Part 4) and **redundancy** (Part 1) exist: so that one machine dying doesn't mean the system dies.
- **Software Fault** — bugs, memory leaks, unhandled edge cases, or a dependency's API changing unexpectedly. Unlike hardware faults, these are often deterministic (the same input reliably triggers the same bug) but can be much harder to detect ahead of time.
- **Network Fault** — packet loss, latency spikes, or a full network partition (directly relevant to the CAP theorem, §4.4) — the network is often the least reliable part of a distributed system, and design must assume messages can be delayed, dropped, or duplicated.

Types of Fault-Tolerance Techniques

- **Redundancy** — running multiple copies of a component (servers, data via replication)

so one failing doesn't take down the system.

- **Failover** — automatically switching to a standby/replica when the primary component fails (e.g., promoting a follower to leader, §4.2).
- **Retries (with backoff)** — automatically re-attempting a failed operation, typically with exponential backoff to avoid overwhelming a struggling downstream service.
- **Timeouts** — giving up on a slow request after a bounded time rather than waiting indefinitely, preventing one slow dependency from cascading into a full system stall.
- **Circuit Breakers** — after repeated failures calling a downstream service, a circuit breaker “trips” and stops sending requests to it for a cooldown period, giving the failing service room to recover instead of being hammered further, and failing fast for callers instead of waiting on a doomed request.
- **Graceful Degradation** — deliberately serving a reduced/partial experience (e.g., showing cached recommendations instead of live ones) rather than a full outage when a dependency is unavailable.

Designing for fault tolerance means assuming failures **will** happen (not if, but when) and combining these techniques so the system degrades gracefully instead of collapsing entirely.

5.4 Monitoring: Latency Percentiles

Simply looking at *average* latency can hide serious problems — a few very slow requests can be masked by many fast ones in an average. This is why systems are monitored using **percentiles**.

- **P (percentile)** — the value below which a given percentage of observations fall.
- **P50 (median)** — half of all requests are faster than this value.
- **P90** — 90% of requests are faster than this value (only the slowest 10% take longer).
- **P99** — 99% of requests are faster than this value (captures the worst 1% “tail latency,” which matters a lot at scale — 1% of a billion requests is still 10 million slow requests).

Example numbers illustrated in the course: an average latency (**K avg**) around **6-8 seconds**, contrasted against percentile breakdowns like **P90 → 12 sec** and **P50 → 4 sec** — showing how the “typical” user experience (median) can look fine while a meaningful slice of users (the tail, captured by P90/P99) has a much worse experience. This is exactly why production systems alert on **P99 latency**, not just the average.

The Three Pillars of Observability (Beyond Latency)

Latency percentiles are one slice of a broader discipline called **observability**, usually described as three complementary pillars:

- **Metrics** — numeric measurements over time (latency percentiles, error rate, CPU

usage, queue depth) — cheap to store and great for dashboards/alerting on trends.

- **Logs** — timestamped, discrete records of individual events (“user 123 failed to log in at 14:02:03”) — great for deep-diving into *what exactly happened* on one specific request.
- **Traces** — follow a single request’s journey across every service it touched (API server → cache → database → downstream service), showing exactly where time was spent — essential for debugging latency in a microservices architecture where one user request might fan out across a dozen internal calls.

Beyond Latency: What Else Gets Monitored

Metrics themselves are typically tracked across several dimensions:

- **Error rate** — percentage of requests returning 4xx/5xx responses.
- **Saturation** — how close a resource (CPU, memory, queue length, connection pool) is to its limit.
- **Traffic** — requests per second, to correlate spikes with latency/error changes.

Together, latency, errors, saturation, and traffic are sometimes called the “**four golden signals**” of monitoring — a useful mental model for any “how would you monitor this system” interview follow-up.

Key Takeaways — Part 5

- Choose sync when the caller genuinely needs to know the outcome before proceeding (payments); choose async (“fire & forget”) when the work can happen independently of the main flow (notifications, emails).
- Message queues are the standard mechanism for turning a synchronous, tightly-coupled dependency into an asynchronous, decoupled one — improving resilience and scalability at the cost of eventual (not immediate) processing.
- Fault tolerance means designing for failure as a certainty, not an edge case — both human faults (unpredictable, process-driven fixes) and hardware faults (redundancy-driven fixes) need explicit mitigation.
- Always monitor percentiles (especially P99), not just averages — averages hide the worst experiences, which are often the ones that drive complaints and churn.

Part 6 — Case Study: Designing a Video Streaming App

This capstone ties together almost every concept from Parts 1-5 into one end-to-end system, similar in spirit to designing YouTube.

6.1 Estimating Requirements

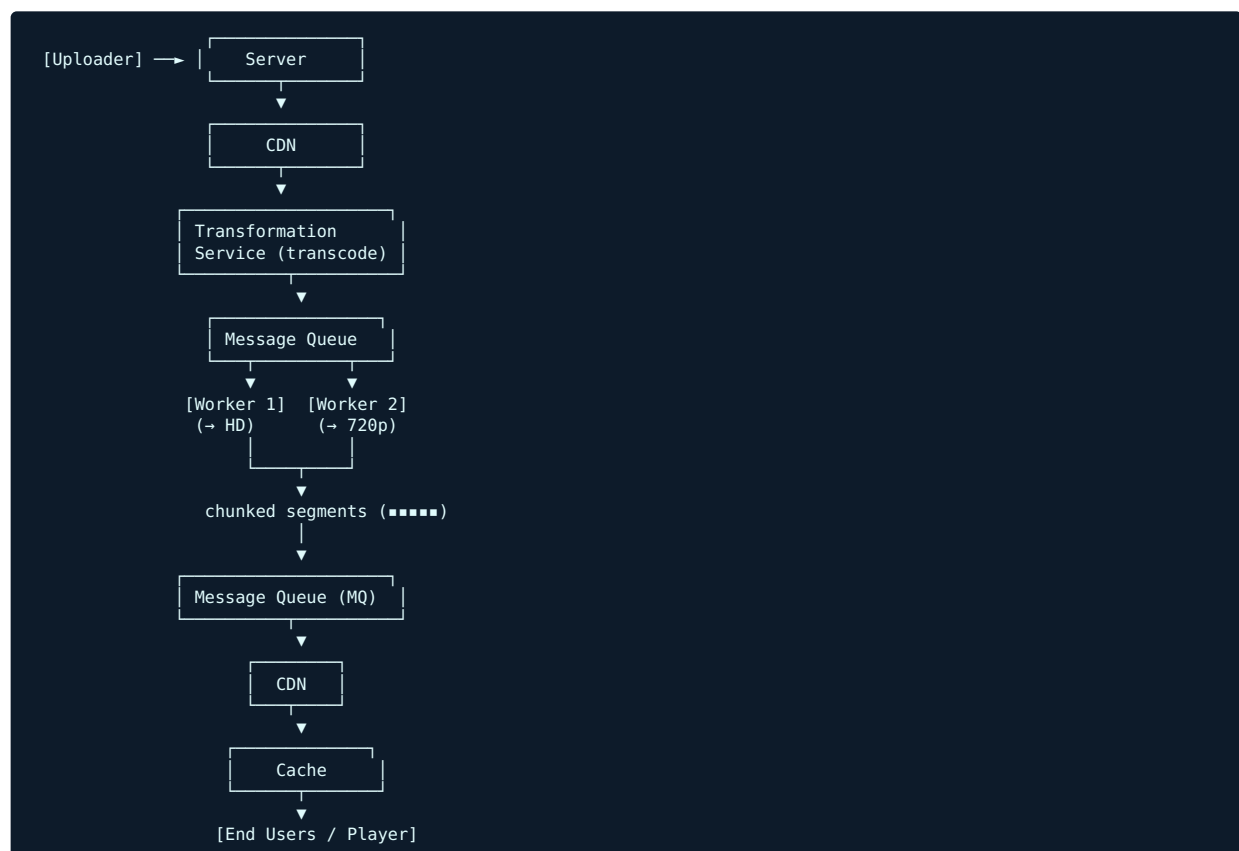
Before designing, the course walks through back-of-envelope **capacity estimation**, a

standard first step in any real system design interview:

- **Different resolutions require different bandwidth.** A comparison is drawn across resolutions: **240p, 480p, 720p, 1080p, up to 4K** — each higher tier needs significantly more data per second.
- Example bandwidth figures used: roughly **300 Mbps** vs **10 Mbps**, contrasting a very high (near-4K/raw) bitrate scenario against a modest one, and a **10 Mbps** figure discussed as a more realistic sustained delivery target per stream for good quality without overwhelming typical user bandwidth.
- **Chunking math:** a video is broken into small segments/chunks for streaming (so playback can start before the whole file downloads, and so different quality levels can be swapped mid-playback — adaptive bitrate streaming). Example shown: a **20-minute** video is converted into seconds ($20 \times 60 = 1200$ seconds) as the basis for calculating how many chunks of a given chunk-length (e.g., a few seconds each) the video will be split into.

This estimation step determines *storage* needs (how many resolutions $\times$ chunk sizes $\times$ total videos) and *bandwidth/CDN* needs (peak concurrent viewers $\times$ bitrate).

6.2 End-to-End Architecture



Walking Through the Flow

1. **Upload:** A creator uploads a raw video file to the **Server**.

2. **CDN (ingest):** The raw upload is pushed toward a **CDN** edge / storage layer close to where it's needed.
3. **Transformation Service:** The raw video is handed to a transcoding pipeline that will convert it into multiple resolutions/formats.
4. **Message Queue:** Rather than transcoding everything in one giant synchronous job, transcoding *tasks* are pushed onto a **Message Queue** — directly reusing the async/decoupling pattern from Part 5.
5. **Worker Pool:** Multiple **Worker** processes pull jobs off the queue in parallel — **Worker 1** produces the **HD** rendition, **Worker 2** produces the **720p** rendition (and, in a full system, additional workers would produce 480p/240p etc.). This is **horizontal scaling** applied to a **compute-intensive** task (transcoding), echoing the compute-intensive vs data-intensive distinction from Part 1.
6. **Chunking:** Each transcoded rendition is split into small playable **segments** (chunks) — enabling adaptive bitrate streaming, where the player can switch quality mid-playback based on the viewer's current network speed.
7. **Second Message Queue stage:** Completed segments flow through another queue stage toward distribution — decoupling transcoding completion from publishing/availability.
8. **CDN (delivery):** Finished, chunked video is pushed out to **CDN** edge nodes positioned close to end users around the world, minimizing latency for playback start and reducing load on the origin server.
9. **Cache:** A caching layer in front of the most-requested (hot) content/metadata — e.g., trending videos — reduces repeated trips back to origin storage, consistent with the caching principles from Part 3.
10. **End Users:** Viewers request the video; their player fetches the appropriate chunk/resolution from the nearest CDN/cache edge, adjusting quality dynamically based on network conditions.

6.3 Why Each Piece Is There (Concept Recap Mapped to This System)

Concern	Concept Used	Where in This Course
Handling upload/transcode spikes without blocking users	Message Queue + async workers	Part 5
Parallel transcoding into multiple resolutions	Horizontal scaling, compute-intensive workloads	Part 1, Part 4
Fast video start / low latency worldwide	CDN + Caching	Part 2 (client-side cache concept), Part 3
Adaptive quality based on network speed	Chunking / segmenting video, multiple resolution renditions	Capacity estimation (6.1)
Surviving a worker or server failure mid-transcode	Fault tolerance, redundancy, replication of workers/queues	Part 4, Part 5

Storing video metadata, comments, user data	SQL (users, transactional data) + NoSQL/Graph (recommendations, relationships)	Part 3
Serving read-heavy traffic (popular videos) without hammering DB	Read-through cache, hot data caching	Part 1.7, Part 3.4
Handling millions of concurrent viewers	Load Balancer across app/API servers	Part 4.1

This case study is a good template to reuse for *any* “design X” interview question (YouTube, Netflix, Instagram, Uber, Dropbox): **estimate scale → draw the request/data flow → plug in load balancing, caching, replication, partitioning, queues, and fault tolerance wherever the estimation step reveals a bottleneck.**

Appendix A — Worked Capacity-Estimation Practice

Back-of-envelope estimation (introduced in §6.1) is a skill best learned by repetition. Two extra worked examples, in the same style as the course’s streaming-app calculation:

Example 1: Storage for a Photo-Sharing App

- Assume **1 million photos uploaded per day**.
- Average photo size (after compression) $\approx$ **2 MB**.
- Daily storage need: $1,000,000 \times 2 \text{ MB} = \mathbf{2,000,000 \text{ MB} \approx 2 \text{ TB/day}}$.
- Over one year: $2 \text{ TB} \times 365 \approx \mathbf{730 \text{ TB/year}}$ — clearly beyond what a single disk or even a single server can hold, which is exactly the justification for **partitioning/sharding** (§4.3) storage across many nodes, plus a **CDN** (§6.2) for fast global delivery.

Example 2: Bandwidth for a Live Chat Feature

- Assume **100,000 concurrent users**, each sending on average **1 message per 10 seconds**.
- Messages per second $\approx 100,000 / 10 = \mathbf{10,000 \text{ messages/sec}}$.
- If each message (with metadata) is $\sim 1 \text{ KB}$, that’s **10 MB/sec** of inbound traffic just for message ingestion — before fan-out to recipients.
- If each message is delivered to an average of 5 recipients (group chats), *outbound* fan-out traffic is roughly $5 \times$ inbound $\approx \mathbf{50 \text{ MB/sec}}$, which immediately motivates a **push-based, queue-backed** delivery pipeline (§5.2) rather than every server trying to synchronously deliver to every recipient.

The pattern to practice: identify the core numbers (users, requests/user, size/request), multiply them out, compare the result to what a single machine can realistically handle, and let *that gap* justify each additional piece of architecture (cache, queue, shard, CDN, replica) you add to the design.

Appendix B — Interview Prep: Questions & Model Answers

Part 1 — Foundations

Q1. What's the difference between functional and non-functional requirements?

Give an example of each for a food-delivery app. Functional requirements describe *what* the system does — e.g., “a user can place an order,” “a restaurant can update its menu.” Non-functional requirements describe *how well* it does it — e.g., “order status must update within 2 seconds” (latency), “the system must stay available during peak dinner-hour traffic” (availability). Interviewers look for candidates who gather both *before* drawing any architecture.

Q2. Why is horizontal scaling generally preferred over vertical scaling for large systems?

Vertical scaling hits a hardware ceiling (there's a maximum CPU/RAM you can put in one box) and keeps a single point of failure. Horizontal scaling distributes load across many commodity machines, has no hard ceiling, and improves fault tolerance — losing one of many machines is recoverable, losing your only machine is not.

Q3. How would you decide if a system is data-intensive or compute-intensive? Ask where the bottleneck actually is: if the challenge is storing, indexing, or moving large volumes of data quickly (a product catalog, a social feed), it's data-intensive. If the challenge is raw processing power per unit of work (video transcoding, ML inference, image processing), it's compute-intensive. Many real systems, like the streaming-app case study, are a mix of both at different stages of the pipeline.

Part 2 — Communication

Q4. Walk through what happens between typing a URL and the page loading.

Browser → ISP → DNS Resolver → Root Server (finds the TLD server) → TLD Server (finds the Authoritative Name Server) → Authoritative Name Server (returns the IP) → browser opens a TCP/TLS connection to that IP → sends an HTTP request → server (behind a load balancer) responds → browser renders the page. Mentioning DNS caching at each layer is a strong signal of depth.

Q5. Why is REST generally preferred over SOAP for modern public APIs? REST typically uses JSON, which is lighter-weight, easier to parse, and maps naturally onto objects in most languages. It's also stateless and resource-oriented, making it simpler to scale horizontally (§4.1) and to cache (§3.4). SOAP's XML payloads and strict contracts (WSDL) add overhead that's rarely justified outside specific enterprise/legacy integration needs.

Q6. What does it mean for an API to be “stateless,” and why does it matter at scale?

Each request carries everything the server needs to process it (auth token, parameters) without relying on server-side memory of previous requests. This means *any* backend server can handle *any* request, which is exactly what makes load balancing without sticky sessions

(§4.1) and horizontal scaling possible.

Part 3 — Data Layer

Q7. When would you choose NoSQL over SQL? When the priority is horizontal write/read scale and flexible/evolving schemas rather than strict transactional guarantees — e.g., a social media feed, a product catalog with wildly varying attributes, session storage, or logging/analytics data. If the data represents money, inventory counts, or anything requiring strict multi-row consistency, SQL's ACID transactions are usually the safer default.

Q8. Explain a Read-Through Cache and what happens on a cache miss. On a read, the backend checks the cache first. On a **hit**, data is returned directly from the cache (fast). On a **miss**, the backend queries the database, returns the result to the caller, *and* writes that result into the cache so the next read for the same key is a hit. This trades a small amount of write overhead on the first read for much faster subsequent reads.

Q9. What's the difference between LRU and LFU eviction policies? LRU (Least Recently Used) evicts the entry that hasn't been *accessed* for the longest time — good for workloads with temporal locality (recently used data tends to be used again soon). LFU (Least Frequently Used) evicts the entry with the *lowest access count* overall — better when some items are consistently popular over long periods regardless of recency.

Part 4 — Scaling & Distribution

Q10. What problem do sticky sessions solve, and what's the modern alternative?

Sticky sessions ensure a user's requests always land on the server holding their in-memory session state, so login/session data stays consistent. The downside is it undermines horizontal scaling and fault tolerance — if that one server dies, the session is lost, and load can't rebalance freely. JWTs solve this by making authentication stateless: the token itself carries verifiable claims, so any server can validate it independently.

Q11. Compare Leader-Follower and Multi-Leader replication. Leader-Follower: all writes go through a single leader, which simplifies consistency (only one node decides the order of writes) but creates a write bottleneck and a failover step if the leader dies. Multi-Leader: multiple nodes accept writes (useful for multi-region latency), which removes the single write bottleneck but introduces the possibility of write **conflicts** when two leaders modify the same data concurrently, requiring a conflict-resolution strategy.

Q12. What is the CAP theorem, and why can't a system have all three properties during a network partition? CAP says a distributed system can only guarantee two of Consistency, Availability, and Partition tolerance at the same time when a network partition occurs. If nodes can't communicate (P, which is unavoidable in real networks), the system must choose: either stop answering some requests until nodes resync (favoring Consistency), or keep answering with potentially stale data (favoring Availability) — it

cannot guarantee both a fully up-to-date answer and a guaranteed answer while the network is split.

Q13. Why do we need both replication and partitioning? Aren't they solving the same problem? No — they solve different problems. **Replication** copies the *same* data onto multiple nodes for durability/availability (if one node dies, another has the same data). **Partitioning** splits *different* data onto different nodes so that no single node has to store or serve the entire dataset. Most large-scale systems use both together: each shard (partition) is itself replicated across multiple nodes.

Part 5 — Reliability & Operations

Q14. Give an example of when you'd use synchronous vs asynchronous communication in the same application. In an e-commerce checkout: charging the customer's card should be **synchronous** — the user needs to know immediately whether payment succeeded before you confirm the order. Sending the "your order has shipped" email, on the other hand, should be **asynchronous** — pushed to a message queue so it doesn't block or slow down the checkout flow, and so a temporary email-service outage doesn't fail the whole order.

Q15. What does a message queue actually buy you, architecturally? Decoupling in both time and failure domain: the producer doesn't need the consumer to be online or fast, and a slow/failing consumer doesn't block the producer. It also smooths out traffic spikes (the queue absorbs a burst; workers drain it at a sustainable rate) and makes it easy to add more consumers to scale processing horizontally.

Q16. Why should you monitor P99 latency instead of just average latency? Averages can look perfectly healthy while a meaningful fraction of users have a terrible experience — a few very slow outliers get smoothed away by many fast requests. P99 explicitly surfaces the worst 1% "tail latency," which matters enormously at scale (1% of a billion daily requests is 10 million bad experiences) and is often what correlates with real user complaints.

Part 6 — Case Study Thinking

Q17. How would you approach "design YouTube/a video streaming app" in an interview, step by step?

1. Clarify functional requirements (upload, transcode, stream, comment, search?) and non-functional priorities (low start-up latency, high availability, huge storage).
2. Do rough capacity estimation (daily uploads × average size, concurrent viewers × bitrate) to reveal the true scale.
3. Sketch the upload → transcode → store → distribute → play pipeline.
4. Apply the toolbox from Parts 3-5: queue-backed async transcoding with parallel

workers (compute-intensive scaling), CDN + caching for playback (read-heavy, latency-sensitive), sharded/replicated storage and metadata databases, load balancers in front of API servers, monitoring on P99 playback start latency.

5. Call out fault tolerance explicitly — what happens if a transcoding worker crashes mid-job, or a CDN edge node goes down.

Q18. Why does the streaming pipeline use a message queue *twice* (once before transcoding, once after)? Each queue stage decouples two independently-scalable, independently-failable pieces of the pipeline: the first queue decouples “video uploaded” from “transcoding capacity,” letting transcoding workers scale up/down and retry failed jobs without blocking new uploads. The second queue decouples “transcoding finished” from “publishing/distribution,” so a slow or temporarily unavailable CDN push doesn’t block workers from picking up the next transcoding job.

Appendix C — Bonus Practice: Applying the Toolkit to a New Problem

(This appendix is supplementary practice, not part of the video itself — it applies the exact concepts from Parts 1–5 to a second classic interview problem, the way you’d be expected to on your own in an interview or exam.)

Design Problem: A URL Shortener (e.g., bit.ly)

Step 1 — Functional Requirements

- Given a long URL, generate a short, unique alias (e.g., `short.ly/abc123`).
- Visiting the short URL redirects to the original long URL.
- (Optional) Track click analytics; allow custom aliases; support link expiry.

Step 2 — Non-Functional Requirements

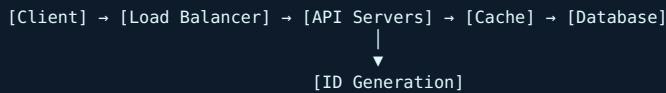
- **Low latency redirects** — this is the hottest path in the whole system; every redirect should feel instant.
- **High availability** — a shortener that’s down breaks every link that depends on it across the web.
- **Read-heavy workload** — redirects (reads) vastly outnumber URL creations (writes), often by 100:1 or more.

Step 3 — Rough Capacity Estimation

- Assume 100 million new short URLs created per month, and a 100:1 read:write ratio → roughly 10 billion redirect reads per month, or on the order of a few thousand reads/second sustained, with much higher peaks.

- Each stored mapping is small (a short code + a long URL + metadata, well under 1 KB), so total storage even at billions of rows is manageable — this is a system dominated by **request volume and latency**, not raw storage size, which immediately tells you: **this problem is about caching and read scaling**, not about sharding for storage capacity.

Step 4 — High-Level Design (mapping straight back to Parts 2-5)



- **API Servers** behind a **Load Balancer** (§4.1) handle both “create short URL” (write) and “redirect” (read) requests — stateless, so no sticky sessions needed.
- **ID Generation**: a common approach is a counter-based or hash-based scheme to produce a short, unique code for each long URL, avoiding collisions without needing a slow global lock.
- **Database**: a simple key-value mapping (short code → long URL) is a textbook fit for a **NoSQL key-value store** (§3.2) — the access pattern is a simple lookup by key, not complex relational queries.
- **Cache** (§3.4): because reads (redirects) dominate so heavily, a **Read-Through Cache** in front of the database is essential — the vast majority of redirect traffic should be served straight from cache, only falling back to the database on a cold/rare short code.
- **Replication** (§4.2): the database should be replicated (Leader-Follower is sufficient here, since writes are relatively infrequent) so redirects keep working even if one database node fails.
- **Partitioning** (§4.3): at very large scale, the key-value store can be partitioned by a hash of the short code, spreading both storage and read load across many nodes.
- **CAP tradeoff** (§4.4): this system should lean **AP** — it’s far better to redirect using a slightly stale replica than to show users an error page because of a momentary consistency check.

Step 5 — Fault Tolerance & Monitoring

- If a database replica fails, traffic fails over to another replica (§4.2, §5.3) with minimal disruption thanks to caching absorbing most read traffic anyway.
- Monitor **P99 redirect latency** (§5.4) as the primary health metric — since this is the system’s entire value proposition to end users.

The takeaway: notice that solving this “new” problem required no new concepts — every decision reused a tool already covered in Parts 2-5. This is exactly the muscle system design interviews are testing: recognizing *which* familiar tool fits *which* part of a new problem.

Appendix D — Common Pitfalls & Anti-Patterns

A collection of mistakes that repeatedly show up in both real systems and interview answers, organized by the part of the course they relate to.

Foundations (Part 1)

- **Jumping straight to architecture diagrams** before clarifying functional and non-functional requirements. Two candidates can draw very different (both “correct”) architectures for the same prompt depending on unstated assumptions about scale — always state your assumptions out loud first.
- **Over-engineering for hypothetical scale.** Not every system needs to be designed for a billion users. Adding sharding, multi-region replication, and a message-queue-per-feature to a system that will realistically serve ten thousand users adds needless complexity and failure surface area.

Communication (Part 2)

- **Treating REST resource design as an afterthought.** Inconsistent, verb-based URLs (`/getUser`, `/createOrder`) make an API harder to cache, harder to secure uniformly, and harder for client teams to reason about compared to consistent, resource-oriented URLs.
- **Ignoring DNS/TLS handshake cost** in latency budgets — for a system where every millisecond counts (e.g., real-time bidding), connection reuse and keep-alive matter as much as the application logic itself.

Data Layer (Part 3)

- **Choosing NoSQL “because it’s more scalable” without checking the access pattern.** If your data is genuinely relational (orders reference customers reference products) and needs multi-row transactional guarantees, forcing it into a NoSQL model often just moves the complexity into application code instead of removing it.
- **Caching everything by default.** A cache adds a second source of truth that can drift from the database (see invalidation, §3.4). Cache what’s genuinely hot/expensive to compute — caching rarely-read data wastes memory and adds invalidation risk for no benefit.
- **No eviction/expiry strategy**, leading to caches that grow unbounded or serve indefinitely stale data.

Scaling & Distribution (Part 4)

- **Defaulting to sticky sessions** instead of stateless auth, silently limiting horizontal scalability and making deployments/failovers riskier.
- **Replicating without a clear failover plan.** Having follower replicas is only half the

solution — you need automated (or well-drilled manual) leader-election/promotion, or replication doesn't actually buy you the availability you think it does.

- **Sharding by the wrong key.** A poorly chosen shard key (e.g., sharding by signup date when most traffic is for recent users) creates “hot shards” that receive disproportionate load, recreating the exact bottleneck sharding was meant to solve.
- **Treating CAP as a one-time, system-wide decision.** In practice, different subsystems of the same product often make different CAP tradeoffs — e.g., an e-commerce site might use a strongly consistent (CP) system for inventory/payment and an eventually consistent (AP) system for product recommendations.

Reliability & Operations (Part 5)

- **Making everything synchronous “to keep it simple.”** This couples unrelated concerns (e.g., a slow email provider now slows down checkout) and removes your ability to absorb traffic spikes.
- **Making everything asynchronous “for scalability.”** Async introduces eventual consistency and operational complexity (dead-letter queues, retries, ordering guarantees); it shouldn't be reached for by default when a user is actively waiting for a definitive answer (e.g., “did my payment go through?”).
- **Alerting only on average latency or uptime,** missing the tail-latency and partial-failure patterns that real users actually experience.

Key Takeaways — Appendix D

Most real-world system design failures are not failures of not knowing a concept — they're failures of applying the *wrong* concept to a given access pattern, or applying a concept *by default* without checking whether the tradeoff actually fits the problem. The discipline of system design is less “memorize more tools” and more “practice matching tool to problem.”

Quick-Reference Glossary

Term	One-Line Definition
HLD / LLD	High-Level Design (architecture) / Low-Level Design (class & code-level design)
Latency	Time for a single request to complete
Throughput	Number of requests handled per unit time
Vertical Scaling	Adding more power to one machine
Horizontal Scaling	Adding more machines
Load Balancer	Distributes traffic across multiple servers
Sticky Session	LB always routes a user to the same server
JWT	Self-contained signed token replacing server-side session state
Replication	Keeping copies of the same data on multiple nodes

Leader-Follower	One node accepts writes, others sync as read replicas
Multi-Leader	Multiple nodes accept writes; risk of conflicts
Partitioning / Sharding	Splitting data across nodes so no single node holds everything
Index	Data structure that speeds up queries on specific columns
CAP Theorem	Pick 2 of Consistency, Availability, Partition tolerance
Cache	Fast storage layer that avoids repeated slow lookups
Read-Through Cache	Backend checks cache before DB; populates cache on miss
LRU / MRU	Evict Least / Most Recently Used cache entry
Sync Communication	Caller waits for response
Async Communication	Caller fires request and continues (“fire & forget”)
Message Queue	Buffer that decouples producers and consumers of work
Fault Tolerance	System keeps working despite component failures
P50 / P90 / P99	Latency percentiles; P99 captures worst-case “tail” latency
CDN	Content Delivery Network — edge servers close to users
REST	Resource-oriented, stateless API style, typically JSON
SOAP	Strict, XML-based API protocol

How to Use These Notes for Interview Prep

1. **Re-derive, don’t memorize.** For each concept, be able to explain *why* it exists (what problem it solves) — that’s how these notes are structured, and it’s how interviewers actually probe understanding.
2. **Practice the estimation step.** Part 6.1’s back-of-envelope math (bandwidth × resolution × users, chunk counts) is a skill in itself — practice it on other systems (chat app, ride-sharing, URL shortener).
3. **Always map a new problem back to the four “worries”** from Part 1.1: read speed, storage safety, concurrent access, machine failure. Every major component in this course (cache, replication, sharding, load balancer, queue, fault tolerance) is a direct answer to one of these four questions.
4. **Use the Part 6.3 table as a checklist** when tackling a new “design X” problem — walk through each row and ask whether that concern applies to your system.